

Лабораторная работа №7

Лебедев Кирилл Дмитриевич 6204-010302D

Задание 1

Был реализован паттерн Iterator.

TabulatedFunction:

// Метод, возвращающий итератор

Iterator<FunctionPoint> iterator();

ArrayTabulatedFunction:

// Реализация метода iterator() с анонимным классом

@Override

public Iterator<FunctionPoint> iterator() {

return new Iterator<FunctionPoint>() {

private int currentIndex = 0;

@Override

public boolean hasNext() {

return currentIndex < pointsCount;

}

@Override

public FunctionPoint next() {

if (!hasNext()) {

throw new NoSuchElementException("Нет следующего элемента");

}

// Возвращаем копию точки для защиты инкапсуляции

FunctionPoint point = points[currentIndex];

currentIndex++;

return new FunctionPoint(point.getX(), point.getY());

}

@Override

public void remove() {

throw new UnsupportedOperationException("Данная операция не поддерживается");

}

};

}

LinkedListTabulatedFunction:

// Реализация метода iterator() с анонимным классом

@Override

public Iterator<FunctionPoint> iterator() {

```

return new Iterator<FunctionPoint>() {
    private FunctionNode currentNode = head.next;
    private int currentIndex = 0;

    @Override
    public boolean hasNext() {
        return currentIndex < pointCounts;
    }

    @Override
    public FunctionPoint next() {
        if (!hasNext()) {
            throw new java.util.NoSuchElementException("No more points available");
        }
        // Возвращаем копию точки для защиты инкапсуляции
        FunctionPoint point = currentNode.point;
        currentNode = currentNode.next;
        currentIndex++;
        return new FunctionPoint(point.getX(), point.getY());
    }

    @Override
    public void remove() {
        throw new UnsupportedOperationException("Remove operation is not
supported");
    }
};
}

```

Код для теста:

```

// Создание функции
TabulatedFunction function = new ArrayTabulatedFunction(0, 10, new double[]{1, 2,
3, 4, 5});

```

```

// Использование в цикле for-e
for (FunctionPoint point : function) {
    println("Point: " + point);
}

```

```

// Или с явным итератором
Iterator<FunctionPoint> iterator = function.iterator();
while (iterator.hasNext()) {

```

```
FunctionPoint point = iterator.next();
println("Point: " + point);
// iterator.remove(); // Выбросит UnsupportedOperationException
}
```

Задание 2

Был реализован паттерн «Фабричный метод».

TabulatedFunctionFactory:

```
package functions;
```

```
public interface TabulatedFunctionFactory {
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int
pointsCount);
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[]
values);
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points);
}
```

Также были добавлены вложенные классы с описанием фабрик, реализующих интерфейс фабрики и порождающие объекты соответствующих классов табулированных функций.

Код для теста:

```
Function fs = new Cos();
TabulatedFunction tf;
```

```
tf = TabulatedFunctions.tabulate(fs, 0, Math.PI, 11);
println(tf.getClass());
```

```
TabulatedFunctions.setTabulatedFunctionFactory(new
LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(fs, 0, Math.PI, 11);
println(tf.getClass());
```

```
TabulatedFunctions.setTabulatedFunctionFactory(new
ArrayTabulatedFunction.ArrayTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(fs, 0, Math.PI, 11);
println(tf.getClass());
```

Задание 3

Была реализована рефлексия.

Методы создания функции через рефлексию:

```
public static TabulatedFunction createTabulatedFunction(Class<?> functionClass,
double leftX, double rightX, int pointsCount) {
    // Проверяем, что класс реализует интерфейс TabulatedFunction
    if (!TabulatedFunction.class.isAssignableFrom(functionClass)) {
        throw new IllegalArgumentException("Класс " + functionClass + " не реализует
интерфейс TabulatedFunction");
    }

    try {
        // Ищем конструктор с параметрами (double, double, int)
        Constructor<?> constructor = functionClass.getConstructor(double.class,
double.class, int.class);
        // Создаем объект с помощью рефлексии
        return (TabulatedFunction) constructor.newInstance(leftX, rightX, pointsCount);
    } catch (Exception e) {
        throw new IllegalArgumentException("Ошибка при создании объекта через
рефлексию", e);
    }
}

public static TabulatedFunction createTabulatedFunction(Class<?> functionClass,
double leftX, double rightX, double[] values) {
    // Проверяем, что класс реализует интерфейс TabulatedFunction
    if (!TabulatedFunction.class.isAssignableFrom(functionClass)) {
        throw new IllegalArgumentException("Класс " + functionClass + " не реализует
интерфейс TabulatedFunction");
    }

    try {
        // Ищем конструктор с параметрами (double, double, double[])
        Constructor<?> constructor = functionClass.getConstructor(double.class,
double.class, double[].class);
        // Создаем объект с помощью рефлексии
        return (TabulatedFunction) constructor.newInstance(leftX, rightX, values);
    } catch (Exception e) {
        throw new IllegalArgumentException("Ошибка при создании объекта через
рефлексию", e);
    }
}
```

```
}
```

```
public static TabulatedFunction createTabulatedFunction(Class<?> functionClass,
FunctionPoint[] points) {
    // Проверяем, что класс реализует интерфейс TabulatedFunction
    if (!TabulatedFunction.class.isAssignableFrom(functionClass)) {
        throw new IllegalArgumentException("Класс " + functionClass + " не реализует
интерфейс TabulatedFunction");
    }

    try {
        // Ищем конструктор с параметрами (FunctionPoint[])
        Constructor<?> constructor =
functionClass.getConstructor(FunctionPoint[].class);
        // Создаем объект с помощью рефлексии
        return (TabulatedFunction) constructor.newInstance((Object) points);
    } catch (Exception e) {
        throw new IllegalArgumentException("Ошибка при создании объекта через
рефлексию", e);
    }
}
```

А также был перегружен метод `tabulate`:

```
// Перегруженные методы tabulate с использованием рефлексии
public static TabulatedFunction tabulate(Class<?> functionClass, Function function, double leftX, double rightX, int pointsCount) {...}

public static TabulatedFunction tabulate(Class<?> functionClass, Function function, double leftX, double rightX, double samplingStep) {...}

// Существующие методы с использованием фабрики (остаются без изменений)
public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int pointsCount) {...}

public static void tabulate(Function function, TabulatedFunction tabulatedFunction) {...}

public static TabulatedFunction tabulate(Function function, double leftX, double rightX, double samplingStep) {...}
```

Код для тестирования:

```
TabulatedFunction f;
```

```
f = TabulatedFunctions.createTabulatedFunction(ArrayTabulatedFunction.class, 0, 10,
3);
```

```
println(f.getClass());
```

```
println(f);
```

```
f = TabulatedFunctions.createTabulatedFunction(ArrayTabulatedFunction.class, 0, 10,
new double[] {0, 10});
```

```
println(f.getClass());  
println(f);
```

```
f = TabulatedFunctions.createTabulatedFunction(LinkedListTabulatedFunction.class,  
    new FunctionPoint[] {  
        new FunctionPoint(0, 0),  
        new FunctionPoint(10, 10)  
    }  
);  
println(f.getClass());  
println(f);
```

```
f = TabulatedFunctions.tabulate(LinkedListTabulatedFunction.class, new Sin(), 0,  
Math.PI, 11);  
println(f.getClass());  
println(f);
```

ВЫВОД В КОНСОЛЬ:

```
Point: (0.0; 1.0)  
Point: (2.5; 2.0)  
Point: (5.0; 3.0)  
Point: (7.5; 4.0)  
Point: (10.0; 5.0)  
Point: (0.0; 1.0)  
Point: (2.5; 2.0)  
Point: (5.0; 3.0)  
Point: (7.5; 4.0)  
Point: (10.0; 5.0)
```

```
class functions.ArrayTabulatedFunction  
class functions.LinkedListTabulatedFunction  
class functions.ArrayTabulatedFunction  
  
class functions.ArrayTabulatedFunction
```

$\{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)\}$

class functions.ArrayTabulatedFunction

$\{(0.0; 0.0), (10.0; 10.0)\}$

class functions.LinkedListTabulatedFunction

$\{(0.0; 0.0), (10.0; 10.0)\}$

class functions.LinkedListTabulatedFunction

$\{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731), (0.9424777960769379; 0.8090169943749475), (1.2566370614359172; 0.9510565162951535), (1.5707963267948966; 1.0), (1.8849555921538759; 0.9510565162951536), (2.199114857512855; 0.8090169943749475), (2.5132741228718345; 0.5877852522924732), (2.827433388230814; 0.3090169943749475), (3.141592653589793; 1.2246467991473532E-16)\}$